Purpose

STD-0 — standards/ Purpose + boundaries

Piece A — Purpose

File: `Axion/libraries/standards/STD-0_purpose.md`

---
library: standards
id: STD-0
schema_version: 1.0.0
status: draft

STD-0 — standards/ Purpose + Boundaries

Purpose
`standards/` defines Axion's **standards pack system**:
- what a standards pack is (structure + metadata)
- how packs are indexed
- how applicability is determined
- how standards are resolved deterministically into a snapshot
- how snapshots are versioned and pinned for reproducibility

Standards are the "rules of build" that shape canonical build, template selection, planning, and verification.

What it governs (in scope)
- Standards pack model (pack_id, scope, rules, constraints)
- Standards index/registry (what packs exist, where they live)
- Applicability mapping (which packs apply to which contexts)
- Resolver order rules (deterministic precedence)
- Resolved standards snapshot format (the output of S3)
- Deprecation rules for packs/standards (optional, recommended)

What it does NOT govern (out of scope)
- The pipeline stage order and stage IO contracts → `orchestration/`
- Gate DSL semantics → `gates/`
- Risk class definitions and overrides → `policy/`
- Intake wizard schema/enums → `intake/`
- Canonical spec schema and unknown handling → `canonical/`
- Templates themselves and render rules → `templates/`
- Knowledge items and selection rules → `knowledge/`

- Verification proof types and completion criteria → `verification/`
- System pins/locks/adapters/quotas → `system/`

Consumers
- Standards resolution stage (S3)
- Canonical builder (S4)
- Template selection (S5)
- Planning (S6)
- Gate evaluation (S8) for standards compliance checks

Determinism requirements
- A standards snapshot is a pure function of:
  - NORMALIZED_INPUT
  - pinned standards index + packs
  - deterministic resolver order rules
  - pinned risk class + profile context
- Snapshots are pinned and recorded in run manifest before downstream stages.

Piece B — Boundary checklist

File: `Axion/libraries/standards/STD-0_boundary_checklist.md`

---
library: standards
id: STD-0a
schema_version: 1.0.0
status: draft

STD-0a — Boundary Checklist

Belongs in `standards/` if it answers:
- “What standards packs exist and what rules do they contain?”
- “Which standards apply for this run context?”
- “How do we resolve conflicting standards deterministically?”
- “What does the resolved snapshot look like?”

Does NOT belong in `standards/` if it answers:
- “How do we evaluate gates?” (gates)
- “What templates do we render?” (templates)
- “What is the canonical spec schema?” (canonical)
- “How is policy invoked?” (system)

Say **“next”** for **STD-1 Standards pack model (pack id, scope, rules)**.

Standards pack model

STD-1 — Standards pack model (pack_id, scope, rules)

Piece A — Standards pack model (concept)

File: `Axion/libraries/standards/STD-1_standards_pack_model.md`

```
---
library: standards
id: STD-1
schema_version: 1.0.0
status: draft
---

# STD-1 — Standards Pack Model

## Purpose
Define what a standards pack is, how it is structured, and how it can be applied deterministically.

## What a standards pack contains
A pack is a versioned, named set of enforceable rules:
- constraints (must/must_not)
- defaults (recommended baseline settings)
- required artifacts (what must be produced)
- verification commands (what must be run)
- policies (naming, layout, dependency boundaries)

## Core fields (minimum)
- pack_id (stable)
- title
- version
- scope (what contexts this pack applies to)
- rules[] (ordered, typed rules)
- created_at, updated_at, owner
```

Piece B — Standards pack schema (JSON Schema)

File: `Axion/libraries/standards/schemas/standards_pack.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
```

```
"id": "standards_pack.v1",
"type": "object",
"required": [
  "pack_id",
  "title",
  "version",
  "scope",
  "rules",
  "created_at",
  "updated_at",
  "owner"
],
"properties": {
  "pack_id": { "type": "string", "pattern": "^STD-[A-Z0-9_\\-]+$" },
  "title": { "type": "string", "minLength": 3 },
  "version": { "type": "string", "minLength": 1 },

  "scope": { "$ref": "#/$defs/scope" },

  "rules": {
    "type": "array",
    "minItems": 1,
    "items": { "$ref": "#/$defs/rule" }
  },

  "created_at": { "type": "string", "format": "date-time" },
  "updated_at": { "type": "string", "format": "date-time" },
  "owner": { "type": "string", "minLength": 2 }
},

"$defs": {
  "scope": {
    "type": "object",
    "required": ["profiles", "risk_classes"],
    "properties": {
      "profiles": {
        "type": "array",
        "items": { "type": "string", "pattern": "^PROFILE-[A-Z0-9_]+$" },
        "minItems": 1
      },
      "risk_classes": {
        "type": "array",
        "items": { "type": "string", "enum": ["PROTOTYPE", "PROD", "COMPLIANCE"] },
        "minItems": 1
```

```
    },
    "stacks": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Optional: technology families (react, node, postgres, etc.)"
    },
    "domains": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Optional: canonical domains (auth, payments, media, etc.)"
    }
  },
  "additionalProperties": false
},

"rule": {
  "type": "object",
  "required": ["rule_id", "type", "severity", "statement"],
  "properties": {
    "rule_id": { "type": "string", "pattern": "^RULE-[A-Z0-9_\\-]+$" },

    "type": {
      "type": "string",
      "enum": ["constraint", "default", "required_artifact", "verification", "naming", "layout"]
    },

    "severity": { "type": "string", "enum": ["must", "should", "must_not"] },

    "statement": { "type": "string", "minLength": 5 },

    "applies_to": {
      "type": "object",
      "description": "Optional narrower applicability within the pack scope",
      "properties": {
        "targets": { "type": "array", "items": { "type": "string" } },
        "entity_types": { "type": "array", "items": { "type": "string" } }
      },
      "additionalProperties": false
    },

    "evidence": {
      "type": "object",
      "description": "Optional: what evidence should exist when checking this rule",
      "properties": {
```

```
          "artifact_contract_ids": { "type": "array", "items": { "type": "string" } },
          "proof_requirements": { "type": "array", "items": { "type": "string" } }
        },
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece C — Standards pack index entry schema (registry points to packs)

File:
`Axion/libraries/standards/schemas/standards_index_entry.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "standards_index_entry.v1",
  "type": "object",
  "required": ["pack_id", "version", "path", "scope", "maturity"],
  "properties": {
    "pack_id": { "type": "string" },
    "version": { "type": "string" },
    "path": { "type": "string" },
    "hash": { "type": "string" },

    "scope": { "type": "object", "additionalProperties": true },

    "maturity": { "type": "string", "enum": ["draft", "reviewed", "verified", "golden"] },
    "deprecated_by": { "type": "string" }
  },
  "additionalProperties": false
}
```

Piece D — Example standards pack (starter)

File: `Axion/libraries/standards/packs/STD-SECURITY_BASELINE.v1.json`


```
{
  "pack_id": "STD-SECURITY_BASELINE",
  "title": "Security Baseline (API/Web)",
  "version": "1.0.0",
  "scope": {
    "profiles": ["PROFILE-API", "PROFILE-WEB"],
    "risk_classes": ["PROTOTYPE", "PROD", "COMPLIANCE"],
    "domains": ["auth", "security"]
  },
  "rules": [
    {
      "rule_id": "RULE-SEC-INPUT-VALIDATION",
      "type": "constraint",
      "severity": "must",
      "statement": "All external inputs must be schema-validated at the boundary before business
logic executes.",
      "evidence": { "artifact_contract_ids": ["NORMALIZED_INPUT"] }
    },
    {
      "rule_id": "RULE-SEC-AUTHZ-OBJECT",
      "type": "constraint",
      "severity": "must",
      "statement": "Authorization must be enforced at object level (resource-specific), not only
route level.",
      "evidence": { "proof_requirements": ["proof:authz_tests_present"] }
    },
    {
      "rule_id": "RULE-SEC-REDACTION",
      "type": "constraint",
      "severity": "must_not",
      "statement": "Secrets, tokens, and credential-bearing headers must not appear in logs or
render envelopes."
    },
    {
      "rule_id": "RULE-SEC-RATE-LIMIT",
      "type": "default",
      "severity": "should",
      "statement": "Auth and privileged endpoints should be rate-limited and have request size
limits."
    },
    {
      "rule_id": "RULE-SEC-VERIFY-AUTH",
```

"type": "verification",
"severity": "should",
"statement": "Include verification commands/tests that demonstrate authN/authZ failures are blocked.",
"evidence": { "proof_requirements": ["proof:auth_tests_run"] }
}
],
"created_at": "2026-03-05T00:00:00Z",
"updated_at": "2026-03-05T00:00:00Z",
"owner": "axion"
}

Piece E — Determinism rules (packs)

File: `Axion/libraries/standards/STD-1_determinism_rules.md`

---
library: standards
id: STD-1a
schema_version: 1.0.0
status: draft

STD-1a — Determinism Rules (Standards Packs)

- pack_id is immutable; pack changes require version bump.
- rules[] order is stable and preserved (used for deterministic diffs and reporting).
- scope fields are closed enums where possible (profile_id, risk_classes).
- Any pack referenced in a snapshot must be pinned by (pack_id, version, path, optional hash).

Piece F — Validation checklist

File: `Axion/libraries/standards/STD-1_validation_checklist.md`

---
library: standards
id: STD-1b
schema_version: 1.0.0
status: draft

STD-1b — Validation Checklist

- [ ] standards packs validate against standards_pack.v1 schema
- [ ] pack_id unique; version present
- [ ] scope profiles reference run_profiles registry (strict mode)
- [ ] rules have rule_id/type/severity/statement
- [ ] rules are ordered deterministically (array order preserved)

Standards index + applicability mapping

STD-2 — Standards index + applicability mapping

Piece A — Standards index model (concept)

File: `Axion/libraries/standards/STD-2_standards_index_model.md`

```
---
library: standards
id: STD-2
schema_version: 1.0.0
status: draft
---

# STD-2 — Standards Index + Applicability Mapping

## Purpose
Provide an authoritative registry of all standards packs and how they apply, so resolution is deterministic.

## Components
1) **STANDARDS_INDEX** (required)
- lists every available pack (pack_id, version, path, scope, maturity, hash optional)

2) **Applicability mapping** (embedded or separate)
- deterministic filters based on:
  - run profile_id (API/WEB/MOBILE/etc.)
  - risk_class
  - optional stack families (react/node/postgres)
  - optional canonical domains (auth/payments/media)

## Design rule
The index is the source-of-truth for “what exists and where it lives.”
No stage should discover packs by directory walking.
```

Piece B — Standards index schema (registry of index entries)

File: `Axion/libraries/standards/schemas/standards_index.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
```

```
  "id": "standards_index.v1",
  "type": "object",
  "required": ["index_id", "schema_version", "packs"],
  "properties": {
    "index_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "packs": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "standards_index_entry.v1.schema.json" }
    }
  },
  "additionalProperties": false
}
```

Piece C — Applicability mapping rules (how to decide what applies)

File: `Axion/libraries/standards/STD-2_applicability_rules.md`

```
---
library: standards
id: STD-2a
schema_version: 1.0.0
status: draft
---

# STD-2a — Applicability Rules

A standards pack applies if ALL are true:
1) run.profile_id is in pack.scope.profiles
2) run.risk_class is in pack.scope.risk_classes
3) If pack.scope.stacks is non-empty:
   - at least one stack matches run.stack_family signals (from normalized_input/canonical meta)
4) If pack.scope.domains is non-empty:
   - at least one domain matches canonical spec domains/capabilities

If a pack omits stacks/domains:
- it is considered “general” within its profiles + risk classes.
```

Piece D — Determinism rules (index + applicability)

File: `Axion/libraries/standards/STD-2_determinism_rules.md`

---
library: standards
id: STD-2b
schema_version: 1.0.0
status: draft

STD-2b — Determinism Rules (Index + Applicability)

- The index is evaluated in listed order.
- Applicability is a pure function of:
  - pinned standards_index
  - pinned packs
  - run profile_id + risk_class
  - canonical/normalized signals
- Packs are never “discovered” by scanning directories.
- If multiple versions of the same pack_id exist in index:
  - resolution rules (STD-3) choose deterministically.

Piece E — Starter standards index (example)

File: `Axion/libraries/standards/registries/standards_index.v1.json`

```
{
  "schema_version": "1.0.0",
  "index_id": "standards_index.v1",
  "packs": [
    {
      "pack_id": "STD-SECURITY_BASELINE",
      "version": "1.0.0",
      "path": "Axion/libraries/standards/packs/STD-SECURITY_BASELINE.v1.json",
      "scope": { "profiles": ["PROFILE-API", "PROFILE-WEB"], "risk_classes": ["PROTOTYPE",
"PROD", "COMPLIANCE"], "domains": ["auth", "security"] },
      "maturity": "draft"
    }
  ]
}
```

Piece F — Validation checklist

File: `Axion/libraries/standards/STD-2_validation_checklist.md`

---
library: standards
id: STD-2c
schema_version: 1.0.0
status: draft

STD-2c — Validation Checklist

- [ ] standards_index validates against standards_index.v1 schema
- [ ] each entry has pack_id/version/path/scope/maturity
- [ ] scope profiles and risk_classes are valid enums (strict mode)
- [ ] applicability rules are deterministic (no directory scanning)

Resolution rules

STD-3 — Resolution rules (deterministic resolver order)

Piece A — Resolution model (concept)

File: `Axion/libraries/standards/STD-3_resolution_model.md`

```
---
library: standards
id: STD-3
schema_version: 1.0.0
status: draft
---

# STD-3 — Standards Resolution Model

## Purpose
When multiple standards packs apply, resolution produces a single ordered “resolved set” with deterministic precedence.

## Resolution outputs
- resolved_pack_list (ordered)
- resolved_rules (ordered)
- conflicts (if any)
- final snapshot artifact (STD-4)

## Inputs
- standards_index (pinned)
- all referenced packs (pinned)
- run context:
  - profile_id
  - risk_class
  - stack signals
  - canonical domains/capabilities
```

Piece B — Resolver order rules (deterministic precedence)

File: `Axion/libraries/standards/STD-3_resolver_order_rules.md`

```
---
library: standards
id: STD-3a
```

schema_version: 1.0.0
status: draft

STD-3a — Resolver Order Rules

Goal
Produce a single deterministic list of packs and rules.

Pack selection
1) Filter packs by applicability (STD-2).
2) If multiple versions of same pack_id are applicable:
  - choose the highest version present in the pinned index (semantic compare)
  - if versions are not semver comparable, use lexicographic compare
3) Sort selected packs by precedence keys:
  1) specificity score (more scope constraints = higher):
    - +2 if scope.domains non-empty
    - +2 if scope.stacks non-empty
    - +1 if scope.profiles length is smaller (more specific)
    - +1 if scope.risk_classes length is smaller (more specific)
  2) maturity (golden > verified > reviewed > draft)
  3) pack_id (lexicographic)

Result: resolved_pack_list (ordered high → low precedence).

Rule layering
Rules are applied in resolved_pack_list order.
When rules conflict:
- must_not beats must beats should
- if equal severity and conflicting statements exist:
  - record a conflict
  - default to most restrictive interpretation
  - if not comparable → require approval (policy) OR fail (risk-class dependent)

Piece C — Conflict schema (optional structured conflict record)

File: `Axion/libraries/standards/schemas/standards_conflict.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "standards_conflict.v1",
```

```
  "type": "object",
  "required": ["conflict_id", "pack_ids", "rule_ids", "summary", "resolution"],
  "properties": {
    "conflict_id": { "type": "string", "pattern": "^STDCONF-[A-Z0-9]{6,}$" },
    "pack_ids": { "type": "array", "items": { "type": "string" }, "minItems": 2 },
    "rule_ids": { "type": "array", "items": { "type": "string" }, "minItems": 2 },
    "summary": { "type": "string", "minLength": 5 },
    "resolution": {
      "type": "object",
      "required": ["mode"],
      "properties": {
        "mode": { "type": "string", "enum": ["most_restrictive", "require_approval", "fail"] },
        "notes": { "type": "string" }
      },
      "additionalProperties": false
    }
  },
  "additionalProperties": false
}
```

Piece D — Determinism rules (resolution)

File: `Axion/libraries/standards/STD-3_determinism_rules.md`

```
---
library: standards
id: STD-3b
schema_version: 1.0.0
status: draft
---

# STD-3b — Determinism Rules (Resolution)

- Applicability filter is deterministic (STD-2).
- Version selection for same pack_id is deterministic.
- Pack precedence sorting uses stable scoring + lexicographic tie-breakers.
- Rule layering is applied in pack precedence order.
- Conflict handling is deterministic using severity ordering:
  must_not > must > should.
- Any unresolved conflict behavior is risk-class + policy controlled and recorded in snapshot.
```

Piece E — Validation checklist

File: `Axion/libraries/standards/STD-3_validation_checklist.md`

```
---
library: standards
id: STD-3c
schema_version: 1.0.0
status: draft
---

# STD-3c — Validation Checklist

- [ ] Resolver order rules defined (specificity + maturity + pack_id)
- [ ] Multiple versions of same pack_id resolved deterministically
- [ ] Conflict handling defined (severity ordering + fallback)
- [ ] Conflicts recorded when encountered (optional schema)
- [ ] Resolution produces stable resolved pack list ordering
```

Snapshot schema

You're already on **STD-3** — the last message I gave *was* **STD-3 (Resolution rules / resolver order)**.

To keep moving **one by one**, the next section is:

STD-4 — Resolved standards snapshot format (output of S3)

Piece A — Snapshot model (concept)

File: `Axion/libraries/standards/STD-4_snapshot_model.md`

```
id: STD-4
library: standards
schema_version: 1.0.0
status: draft

# STD-4 — Resolved Standards Snapshot Model

## Purpose
A standards snapshot is the deterministic output of S3. It is the frozen set of standards packs
and rules that downstream stages must follow.

## What the snapshot must contain
- snapshot_id (stable)
- run_id
- resolver_version (version of the resolver logic)
- resolved_packs[] (ordered list: pack_id + version + path + hash optional)
- resolved_rules[] (ordered list: expanded rules with pack source)
- conflicts[] (if any)
- effective_settings (derived defaults, if you compute them)
- created_at

## Design constraints
- Everything in the snapshot must be reproducible from pinned inputs.
- Ordering is deterministic (matches STD-3 resolved order).
- Snapshot is pinned in the run manifest and treated as read-only.
```

Piece B — Snapshot schema (JSON Schema)

File: `Axion/libraries/standards/schemas/standards_snapshot.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "standards_snapshot.v1",
  "type": "object",
  "required": [
    "snapshot_id",
    "run_id",
    "resolver_version",
    "resolved_packs",
    "resolved_rules",
    "created_at"
  ],
  "properties": {
    "snapshot_id": { "type": "string", "pattern": "^STDSNAP-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string" },
    "resolver_version": { "type": "string", "minLength": 1 },

    "resolved_packs": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["pack_id", "version", "path"],
        "properties": {
          "pack_id": { "type": "string" },
          "version": { "type": "string" },
          "path": { "type": "string" },
          "hash": { "type": "string" },
          "maturity": { "type": "string", "enum": ["draft", "reviewed", "verified", "golden"] }
        },
        "additionalProperties": false
      }
    },

    "resolved_rules": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["pack_id", "pack_version", "rule_id", "type", "severity", "statement"],
        "properties": {
```

```
        "pack_id": { "type": "string" },
        "pack_version": { "type": "string" },
        "rule_id": { "type": "string" },
        "type": { "type": "string" },
        "severity": { "type": "string" },
        "statement": { "type": "string" },
        "applies_to": { "type": "object", "additionalProperties": true },
        "evidence": { "type": "object", "additionalProperties": true }
      },
      "additionalProperties": false
    }
  },

  "conflicts": {
    "type": "array",
    "items": { "type": "object", "additionalProperties": true }
  },

  "effective_settings": {
    "type": "object",
    "description": "Optional derived defaults/settings merged from packs",
    "additionalProperties": true
  },

  "created_at": { "type": "string", "format": "date-time" }
 },
 "additionalProperties": false
}
```

Piece C — Determinism rules (snapshot)

File: `Axion/libraries/standards/STD-4_determinism_rules.md`

id: STD-4a
library: standards
schema_version: 1.0.0
status: draft

STD-4a — Determinism Rules (Snapshot)

- resolved_packs order must equal STD-3 resolved pack list.
- resolved_rules order must equal:

pack precedence order, then rule array order inside each pack.
- snapshot_id is generated deterministically from:
pipeline version + resolved pack refs + run_id (or from pack refs only, if you want cross-run stable snapshots).
- conflicts (if present) must be ordered deterministically by conflict_id (or by pack_id/rule_id pairs).
- Snapshot is pinned in run manifest and treated as read-only.

Piece D — Validation checklist

File: `Axion/libraries/standards/STD-4_validation_checklist.md`

id: STD-4b
library: standards
schema_version: 1.0.0
status: draft

STD-4b — Validation Checklist

- [ ] standards_snapshot validates against schema
- [ ] resolved_packs is non-empty and ordered deterministically
- [ ] resolved_rules is non-empty and ordered deterministically
- [ ] every resolved_rule references a pack_id + pack_version present in resolved_packs
- [ ] conflicts (if present) are deterministic and referenced in snapshot notes/evidence

Gates

STD-5 — Gates (resolved, complete, pinned)

Piece A — Standards gates intent

File: `Axion/libraries/standards/STD-5_standards_gates.md`

```
---
library: standards
id: STD-5
schema_version: 1.0.0
status: draft
---

# STD-5 — Standards Gates

## Purpose
Ensure standards resolution (S3) produced a valid, deterministic snapshot and that downstream stages are operating on pinned standards.

## Gate set (minimum)

### STD-GATE-01 — Standards index pinned + valid
- standards_index is pinned (via SYS pins)
- standards_index validates against standards_index.v1

### STD-GATE-02 — All referenced packs exist + validate
- each selected pack path exists
- each pack validates against standards_pack.v1

### STD-GATE-03 — Resolver produced snapshot
- STANDARDS_SNAPSHOT exists
- snapshot validates against standards_snapshot.v1

### STD-GATE-04 — Snapshot pack ordering deterministic
- snapshot.resolved_packs matches STD-3 resolver output order
- snapshot.resolved_rules ordering is deterministic (pack order → rule order)

### STD-GATE-05 — Conflicts handled deterministically
- if conflicts exist:
  - they are recorded in snapshot.conflicts
  - resolution mode is deterministic (most_restrictive / require_approval / fail)
  - for require_approval: policy hook decision exists in run manifest
```

STD-GATE-06 — Snapshot pinned in run manifest
- run manifest includes artifact entry for STANDARDS_SNAPSHOT
- downstream stages reference the pinned snapshot (not “latest”)

Piece B — Mapping to pipeline gate (G3_STANDARDS_RESOLVED)

File: `Axion/libraries/standards/STD-5_gate_mapping.md`

```
---
library: standards
id: STD-5a
schema_version: 1.0.0
status: draft
---

# STD-5a — Mapping to Pipeline Gate

Pipeline gate: **G3_STANDARDS_RESOLVED**

G3 passes if:
- STD-GATE-01..04 pass
- STD-GATE-05 passes (or is policy-approved when required)
- STD-GATE-06 passes

Default behavior:
- PROTOTYPE: conflicts may warn/pause (configurable)
- PROD: conflicts require approval or fail
- COMPLIANCE: conflicts fail by default
```

Piece C — Machine-readable gate spec (optional)

File: `Axion/libraries/standards/STD-5_standards_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "STD-5b",
  "gates": [
    { "gate_id": "STD-GATE-01", "name": "Standards index pinned + valid", "severity":
"hard_stop", "checks": ["pinned(standards_index)", "schema_valid(standards_index.v1)"] },
```

```
    { "gate_id": "STD-GATE-02", "name": "Referenced packs exist + validate", "severity":
"hard_stop", "checks": ["all selected pack paths exist", "schema_valid(standards_pack.v1)"] },
    { "gate_id": "STD-GATE-03", "name": "Resolver produced snapshot", "severity": "hard_stop",
"checks": ["STANDARDS_SNAPSHOT exists", "schema_valid(standards_snapshot.v1)"] },
    { "gate_id": "STD-GATE-04", "name": "Snapshot ordering deterministic", "severity":
"hard_stop", "checks": ["resolved_packs ordered", "resolved_rules ordered"] },
    { "gate_id": "STD-GATE-05", "name": "Conflicts handled deterministically", "severity":
"policy_controlled", "checks": ["conflicts recorded and resolved deterministically"] },
    { "gate_id": "STD-GATE-06", "name": "Snapshot pinned in run manifest", "severity":
"hard_stop", "checks": ["manifest references STANDARDS_SNAPSHOT artifact"] }
  ]
}
```

Piece D — Evidence requirements

File: `Axion/libraries/standards/STD-5_evidence_requirements.md`

```
---
library: standards
id: STD-5c
schema_version: 1.0.0
status: draft
---

# STD-5c — Evidence Requirements (Standards Failures)

On failure, evidence must include:
- pointer to standards_index entry (pack_id/version/path)
- pointer to missing/invalid pack file (path)
- pointer to snapshot fields (resolved_packs/resolved_rules/conflicts)
- conflict details:
  - pack_ids + rule_ids
  - chosen resolution mode
  - policy decision ref if approval required
- remediation:
  - fix pack schema
  - fix applicability/scope
  - resolve conflict or approve override
  - rerun S3 and downstream stages
```

Piece E — Determinism rules

File: `Axion/libraries/standards/STD-5_determinism_rules.md`

```
---
library: standards
id: STD-5d
schema_version: 1.0.0
status: draft
---

# STD-5d — Determinism Rules (Standards Gates)

- Gates run in fixed order STD-GATE-01..06.
- Output is stable given pinned inputs.
- Policy-controlled outcomes must reference pinned policy_set_id + decision_id.
- Evidence pointers are deterministic and path-based (hash optional).
```

Piece F — Validation checklist

File: `Axion/libraries/standards/STD-5_validation_checklist.md`

```
---
library: standards
id: STD-5e
schema_version: 1.0.0
status: draft
---

# STD-5e — Validation Checklist

- [ ] Standards gates cover index pinning, pack validation, snapshot presence, ordering,
conflicts, manifest pinning
- [ ] G3 mapping is explicit
- [ ] Conflicts handled deterministically and policy decisions recorded when required
- [ ] Evidence pointers + remediation included
```

Minimum viable standards

STD-6 — Minimum viable standards set (runtime-required files + “done”)

Piece A — MV standards inventory

File: `Axion/libraries/standards/STD-6_minimum_viable_set.md`

---
library: standards
id: STD-6
schema_version: 1.0.0
status: draft

STD-6 — Minimum Viable standards/ Set

Goal
Define the smallest standards library set required to resolve a standards snapshot deterministically.

Required runtime files (minimum)
1) Core docs
- STD-0_purpose.md
- STD-0_boundary_checklist.md
- STD-1_standards_pack_model.md
- STD-1_determinism_rules.md
- STD-1_validation_checklist.md
- STD-2_standards_index_model.md
- STD-2_applicability_rules.md
- STD-2_determinism_rules.md
- STD-2_validation_checklist.md
- STD-3_resolution_model.md
- STD-3_resolver_order_rules.md
- STD-3_determinism_rules.md
- STD-3_validation_checklist.md
- STD-4_snapshot_model.md
- STD-4_determinism_rules.md
- STD-4_validation_checklist.md
- STD-5_standards_gates.md
- STD-5_gate_mapping.md
- STD-5_evidence_requirements.md
- STD-5_determinism_rules.md

- STD-5_validation_checklist.md
- STD-6_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/standards_pack.v1.schema.json
- schemas/standards_index_entry.v1.schema.json
- schemas/standards_index.v1.schema.json
- schemas/standards_snapshot.v1.schema.json
- schemas/standards_conflict.v1.schema.json (optional)

3) Registries (runtime defaults)
- registries/standards_index.v1.json
- packs/<pack files referenced by index> (at least one pack for testing)
- registries/standards_resolver_version.v1.json (optional, if you pin resolver version explicitly)

4) Gate specs (optional)
- STD-5_standards_gates.spec.json

Piece B — Definition of done (standards/)

File: `Axion/libraries/standards/STD-6_definition_of_done.md`

---
library: standards
id: STD-6a
schema_version: 1.0.0
status: draft

STD-6a — standards/ Definition of Done

standards/ is “done” when:

Schemas + registries
- [ ] All standards schemas validate (JSON Schema check)
- [ ] standards_index validates and references existing pack paths
- [ ] all referenced pack files validate against standards_pack schema

Runtime behavior (contract-level)
- [ ] S3 selects applicable packs deterministically (STD-2)
- [ ] S3 resolves pack precedence deterministically (STD-3)
- [ ] S3 emits standards_snapshot that validates (STD-4)

- [ ] Snapshot ordering matches resolver ordering
- [ ] Conflicts are recorded and resolved deterministically (or policy approved)

Gates
- [ ] STD-GATE-01..06 implemented and mapped to G3_STANDARDS_RESOLVED
- [ ] Snapshot pinned in run manifest before downstream stages

---

Piece C — Minimal folder tree

File: `Axion/libraries/standards/STD-6_minimal_tree.txt`

```
Axion/libraries/standards/
  STD-0_purpose.md
  STD-0_boundary_checklist.md
  STD-1_standards_pack_model.md
  STD-1_determinism_rules.md
  STD-1_validation_checklist.md
  STD-2_standards_index_model.md
  STD-2_applicability_rules.md
  STD-2_determinism_rules.md
  STD-2_validation_checklist.md
  STD-3_resolution_model.md
  STD-3_resolver_order_rules.md
  STD-3_determinism_rules.md
  STD-3_validation_checklist.md
  STD-4_snapshot_model.md
  STD-4_determinism_rules.md
  STD-4_validation_checklist.md
  STD-5_standards_gates.md
  STD-5_gate_mapping.md
  STD-5_evidence_requirements.md
  STD-5_determinism_rules.md
  STD-5_validation_checklist.md
  STD-6_minimum_viable_set.md
  STD-6_definition_of_done.md
  STD-6_minimal_tree.txt

  schemas/
    standards_pack.v1.schema.json
    standards_index_entry.v1.schema.json
    standards_index.v1.schema.json
    standards_snapshot.v1.schema.json
```

```
  standards_conflict.v1.schema.json

registries/
  standards_index.v1.json

packs/
  STD-SECURITY_BASELINE.v1.json
```

Say **“next”** to start the next library folder. Recommended next is **templates/** (TMP-0).